Banco de situaciones TypeScript + React

Más de 380 situaciones para practicar, de menor a mayor dificultad.
Basado en tu guía (PDF) y en el ensayo UT01-02 "Elección de delegado/a".

Cómo usarlo

1. Lee el **enunciado** de cada situación (el título) e intenta escribir la solución tú antes de mirar el código.
2. Marca ✓ las que te salen solas y ↻ las que te cuestan. Repite las ↻ al día siguiente.
3. Para comprobar tipos: `npx tsc --noEmit`.
4. Niveles: ★ básico · ★★ medio · ★★★ avanzado.

Tu nivel según lo que has estudiado

Por el PDF y el ensayo, dominas (o estás a punto de dominar) lo básico de React con TS: props tipadas, `useState`, `map` + `key`, formulario controlado, `preventDefault`, elevar estado, inmutabilidad con `map`/`filter`/spread y datos derivados. Lo que más te va a ayudar a subir de nivel, y que este banco trabaja mucho, es:

- **Manipular arrays con soltura**: ordenar (asc, desc, alfabético, precio, categoría, varios criterios), filtrar, agrupar, reducir.
- **Narrowing y uniones** (`undefined`, `typeof`, uniones discriminadas).
- **Genéricos y utility types**.
- **Variantes del mismo ejercicio** (el examen suele cambiar un detalle del enunciado que ya conoces).

Datos base (se usan en muchas situaciones)

```
type Producto = {
  id: number;
  nombre: string;
  precio: number;
  categoria: string;
  stock: number;
  fecha: string; // formato ISO "AAAA-MM-DD"
};

const productos: Producto[] = [
  { id: 1, nombre: "Ratón",      precio: 15,  categoria: "Periféricos", stock: 12, fecha: "2026-03-10" },
  { id: 2, nombre: "teclado",    precio: 30,  categoria: "Periféricos", stock: 0,  fecha: "2026-01-05" },
  { id: 3, nombre: "Monitor",    precio: 180, categoria: "Pantallas",   stock: 4,  fecha: "2026-05-22" },
  { id: 4, nombre: "Cable HDMI", precio: 8,   categoria: "Cables",      stock: 50, fecha: "2025-11-30" },
  { id: 5, nombre: "Escritorio", precio: 120, categoria: "Muebles",     stock: 3,  fecha: "2026-02-14" },
  { id: 6, nombre: "Silla",      precio: 95,  categoria: "Muebles",     stock: 7,  fecha: "2026-04-01" },
  { id: 7, nombre: "Webcam",     precio: 45,  categoria: "Periféricos", stock: 9,  fecha: "2026-03-10" },
];

type Candidato = {
  id: number;
  nombre: string;
  numeroLista: number;
  votos: number;
  lema?: string;
};
```

```ts
const candidatos: Candidato[] = [
  { id: 1, nombre: "Lucía",  numeroLista: 4,  votos: 7, lema: "Más recreos" },
  { id: 2, nombre: "Álvaro", numeroLista: 12, votos: 3 },
  { id: 3, nombre: "Marta",  numeroLista: 2,  votos: 7, lema: "Clase unida" },
  { id: 4, nombre: "Iván",   numeroLista: 9,  votos: 0 },
];

const nombres = ["Zoe", "álvaro", "Ana", "Ñandú", "ángel", "Óscar", "bea"];
const numeros = [10, 9, 1, 100, 25, 3];
```

En los ejemplos de React se omiten los `import` de `useState`, `useEffect`, etc. Acuérdate de ponerlos.

---

1. Tipos básicos, inferencia y alias

1.1 ★ Declara una variable de cada tipo primitivo

```ts
let nota: number = 8.5;
let nombre: string = "Ana";
let aprobado: boolean = true;
```

1.2 ★ ¿Qué tipo infiere TypeScript?

```ts
let puntos = 10;        // number
const ciudad = "Oviedo"; // tipo literal "Oviedo"
let activo = false;     // boolean
// puntos = "diez";     // Error
```

1.3 ★ Un array de strings y otro de números

```ts
const frutas: string[] = ["pera", "manzana"];
const notas: Array<number> = [7, 8.5, 10];
```

1.4 ★ Array vacío que luego se llena

```ts
const tareas: string[] = [];   // sin tipo sería never[]
tareas.push("estudiar");
```

1.5 ★ Una tupla (longitud y tipos fijos)

```ts
let par: [string, number] = ["edad", 22];
const [clave, valor] = par;
```

1.6 ★ Un objeto con tipo propio

```ts
type Alumno = { nombre: string; edad: number; matriculado: boolean };
const a: Alumno = { nombre: "Isaac", edad: 20, matriculado: true };
```

1.7 ★ Propiedad opcional (como `lema?`)

```ts
type Usuario = { nombre: string; edad?: number };
const u1: Usuario = { nombre: "Ana" };
const u2: Usuario = { nombre: "Luis", edad: 30 };
```

1.8 ★ Propiedad de solo lectura

```ts
type Config = { readonly version: string; modo: string };
const c: Config = { version: "1.0", modo: "dev" };
// c.version = "2.0"; // Error
```

1.9 ★ Unión de tipos

```ts
let codigo: string | number = "ID-402";
codigo = 402;
```

1.10 ★ Tipo literal con tres valores (como `Estado`)

```ts
type Estado = "ninguno" | "algunos" | "lider";
let e: Estado = "lider";
// e = "otro"; // Error
```

1.11 ★ Un tipo que admite `null`

```ts
let seleccionado: Candidato | null = null;
seleccionado = candidatos[0];
```

1.12 ★ `any` frente a `unknown`

```
let a: any = 10;
a.loQueSea();            // no da error (mal)
let u: unknown = "hola";
// u.length;             // Error: hay que comprobar antes
if (typeof u === "string") console.log(u.length);
```

1.13 ★ Función que no devuelve nada (`void`)

```
function avisar(mensaje: string): void {
  console.log(mensaje);
}
```

1.14 ★★ Función que nunca termina (`never`)

```
function fallar(mensaje: string): never {
  throw new Error(mensaje);
}
```

1.15 ★ `interface` equivalente a un `type`

```
interface Persona { nombre: string; edad: number }
```

1.16 ★★ Ampliar una interface con `extends`

```
interface Persona { nombre: string }
interface Delegado extends Persona { curso: string }
```

1.17 ★★ Intersección de tipos con `&`

```
type ItemCarrito = Producto & { cantidad: number };
```

1.18 ★★ Diccionario con `Record`

```
const votosPorClase: Record<string, number> = { "1A": 12, "1B": 9 };
votosPorClase["2A"] = 15;
```

1.19 ★★ Firma de índice (alternativa a `Record`)

```
type Notas = { [asignatura: string]: number };
const n: Notas = { DIW: 8, DAW: 9 };
```

1.20 ★★ Lista cerrada de valores con `as const`

```
const CATEGORIAS = ["Periféricos", "Pantallas", "Cables", "Muebles"] as const;
type Categoria = (typeof CATEGORIAS)[number]; // "Periféricos" | "Pantallas" | ...
```

1.21 ★★ `enum` y su alternativa moderna

```
enum Rol { Alumno, Profesor }          // genera código JS
type RolU = "alumno" | "profesor";     // más ligera, suele preferirse
```

1.22 ★★ Sacar el tipo de una propiedad con índice

```
type Precio = Producto["precio"];      // number
type Claves = keyof Producto;          // "id" | "nombre" | "precio" | ...
```

1.23 ★★ Obtener el tipo a partir de un valor con `typeof`

```
const configuracion = { tema: "oscuro", tamano: 14 };
type Configuracion = typeof configuracion; // { tema: string; tamano: number }
```

1.24 ★★ Aserción de tipo (úsala con cuidado)

```
const entrada = document.getElementById("nombre") as HTMLInputElement | null;
```

1.25 ★★ Tipo de un elemento de array

```
type Elemento = (typeof productos)[number]; // Producto
```

1.26 ★ Tipo para una función

```
type Operacion = (a: number, b: number) => number;
const sumar: Operacion = (a, b) => a + b;
```

1.27 ★ Un array de objetos

```
const lista: Candidato[] = [];
const otra: Array<{ id: number; texto: string }> = [];
```

1.28 ★★ Tipo para un callback que avisa al padre (como `alVotar`)

```
type AlVotar = (id: number) => void;
type AlAnadir = (c: Candidato) => void;
```

1.29 ★ Constante literal vs variable

```
const modo = "oscuro";   // tipo "oscuro"
let modo2 = "oscuro";    // tipo string
```

1.30 ★★ Tipo fecha: string ISO o `Date`

```
type Evento = { titulo: string; cuando: string | Date };
```

---

2. Narrowing (comprobar antes de usar)

2.1 ★ Comprobar `undefined` antes de operar

```
function siguienteEdad(u: Usuario): number | string {
  if (u.edad !== undefined) return u.edad + 1;
  return "sin edad";
}
```

2.2 ★ `typeof` para acotar `string | number`

```
function mostrar(v: string | number) {
  if (typeof v === "string") return v.toUpperCase();
  return v.toFixed(2);
}
```

2.3 ★ Encadenamiento opcional `?.`

```
const longitudLema = candidatos[1].lema?.length;   // number | undefined
```

2.4 ★ Valor por defecto con `??`

```
const lema = candidatos[1].lema ?? "Sin lema";
```

2.5 ★★ Diferencia entre `||` y `??`

```
const votos = 0;
const a = votos || 10;  // 10  (0 es "falso")
const b = votos ?? 10;  // 0   (solo null/undefined)
```

2.6 ★★ Operador `in`

```
type Gato = { maullar: () => void };
type Perro = { ladrar: () => void };
function sonido(m: Gato | Perro) {
  if ("maullar" in m) m.maullar();
  else m.ladrar();
}
```

2.7 ★★ Unión discriminada

```
type Resultado =
  | { tipo: "ok"; datos: string[] }
  | { tipo: "error"; mensaje: string };

function pintar(r: Resultado) {
  if (r.tipo === "ok") return r.datos.join(", ");
  return r.mensaje;
}
```

2.8 ★★ `switch` sobre una unión

```
function etiqueta(e: Estado): string {
  switch (e) {
    case "ninguno": return "Sin votos";
    case "algunos": return "Con votos";
    case "lider":   return "♛ Líder";
  }
}
```

2.9 ★★★ Comprobación exhaustiva con `never`

```
function etiqueta2(e: Estado): string {
  switch (e) {
    case "ninguno": return "Sin votos";
    case "algunos": return "Con votos";
    case "lider":   return "Líder";
    default: {
      const _comprobar: never = e; // si añades un valor a Estado, esto da error
      return _comprobar;
    }
  }
}
```

2.10 ★★ Guarda de tipo personalizada

```
function esString(x: unknown): x is string {
  return typeof x === "string";
}
const mezcla: unknown[] = ["a", 1, "b"];
const soloTextos: string[] = mezcla.filter(esString);
```

2.11 ★★ `Array.isArray`

```
function aLista(x: string | string[]): string[] {
  return Array.isArray(x) ? x : [x];
}
```

2.12 ★★ `instanceof`

```
function mensaje(err: unknown): string {
  return err instanceof Error ? err.message : "Error desconocido";
}
```

2.13 ★★ Operador `!` (aserción no nula): cuándo evitarlo

```
const primero = candidatos.find(c => c.id === 1)!; // si no existe, revienta
// Mejor:
const p = candidatos.find(c => c.id === 1);
if (p) console.log(p.nombre);
```

2.14 ★ Comprobar que un string no está vacío

```
function tieneTexto(s: string): boolean {
  return s.trim().length > 0;
}
```

2.15 ★★ Acotar un `null` antes de usar

```
function nombreDe(c: Candidato | null): string {
  if (c === null) return "nadie";
  return c.nombre;
}
```

---

3. Funciones tipadas

3.1 ★ Parámetros y retorno tipados

```
const multiplicar = (a: number, b: number): number => a * b;
```

3.2 ★ Parámetro opcional

```
function saludar(nombre: string, titulo?: string): string {
  return titulo ? `${titulo} ${nombre}` : nombre;
}
```

3.3 ★ Parámetro con valor por defecto

```
function precioConIva(precio: number, iva: number = 0.21): number {
  return precio * (1 + iva);
}
```

3.4 ★★ Parámetros rest

```
function sumarTodos(...numeros: number[]): number {
  return numeros.reduce((s, n) => s + n, 0);
}
sumarTodos(1, 2, 3); // 6
```

3.5 ★ `obtenerEstado` (el del ensayo)

```
function obtenerEstado(votos: number, maximo: number): Estado {
  if (votos === 0) return "ninguno";
  if (votos === maximo) return "lider";
  return "algunos";
}
```

3.6 ★★ `obtenerEstado` con `switch (true)`

```
function obtenerEstado2(votos: number, maximo: number): Estado {
  switch (true) {
    case votos === 0: return "ninguno";
    case votos === maximo: return "lider";
    default: return "algunos";
  }
}
```

3.7 ★★ Variante: añadir un cuarto estado `"empate"`

```
type Estado4 = "ninguno" | "algunos" | "lider" | "empate";
function obtenerEstado4(votos: number, maximo: number, cuantosConMaximo: number): Estado4 {
  if (votos === 0) return "ninguno";
  if (votos === maximo) return cuantosConMaximo > 1 ? "empate" : "lider";
  return "algunos";
}
```

3.8 ★ Clasificar una nota

```
function calificacion(n: number): "suspenso" | "aprobado" | "notable" | "sobresaliente" {
  if (n < 5) return "suspenso";
  if (n < 7) return "aprobado";
  if (n < 9) return "notable";
  return "sobresaliente";
}
```

3.9 ★ Formatear un precio

```
const formatearPrecio = (p: number): string => p.toFixed(2).replace(".", ",") + " €";
// formatearPrecio(9.5) → "9,50 €"
```

3.10 ★★ Formatear con `Intl.NumberFormat`

```
const eur = new Intl.NumberFormat("es-ES", { style: "currency", currency: "EUR" });
eur.format(1234.5); // "1234,50 €" (con separador de miles según el navegador)
```

3.11 ★ Validar que un número de lista está entre 1 y 30

```
function numeroListaValido(texto: string): boolean {
  if (texto.trim() === "") return false;
  const n = Number(texto);
  return Number.isInteger(n) && n >= 1 && n <= 30;
}
```

3.12 ★★ Convertir texto a número devolviendo `number | null`

```
function aNumero(texto: string): number | null {
  const n = Number(texto);
  return texto.trim() === "" || Number.isNaN(n) ? null : n;
}
```

3.13 ★ Capitalizar la primera letra

```
const capitalizar = (s: string): string =>
  s.length === 0 ? s : s[0].toUpperCase() + s.slice(1);
```

3.14 ★ Contar palabras

```
const contarPalabras = (s: string): number =>
  s.trim() === "" ? 0 : s.trim().split(/\s+/).length;
```

3.15 ★ Validar un email sencillo

```
const emailValido = (e: string): boolean => /^\S+@\S+\.\S+$/.test(e);
```

3.16 ★★ Función que devuelve otra función

```
const multiplicadorDe = (factor: number) => (n: number): number => n * factor;
const doble = multiplicadorDe(2);
doble(5); // 10
```

3.17 ★★ Función que recibe un callback

```
function paraCada(lista: number[], accion: (n: number) => void): void {
  for (const n of lista) accion(n);
}
```

3.18 ★★ Tipar el retorno como objeto

```
function estadisticas(lista: number[]): { min: number; max: number; media: number } {
  const suma = lista.reduce((s, n) => s + n, 0);
  return { min: Math.min(...lista), max: Math.max(...lista), media: suma / lista.length };
}
```

3.19 ★★ Sobrecarga de funciones

```
function duplicar(x: string): string;
function duplicar(x: number): number;
function duplicar(x: string | number) {
  return typeof x === "string" ? x + x : x * 2;
}
```

3.20 ★ Calcular el máximo de votos (apartado 3 del ensayo)

```
const maximo = Math.max(0, ...candidatos.map(c => c.votos));
```

3.21 ★ Calcular el total de votos

```
const totalVotos = candidatos.reduce((suma, c) => suma + c.votos, 0);
```

3.22 ★★ Porcentaje de votos de un candidato (sin dividir entre 0)

```
const porcentaje = (votos: number, total: number): number =>
  total === 0 ? 0 : Math.round((votos / total) * 100);
```

3.23 ★ Fecha de hoy en formato ISO

```
const hoy: string = new Date().toISOString().slice(0, 10);
```

3.24 ★★ Función pura frente a función con efecto

```
// Pura: mismo resultado para los mismos datos, no toca nada de fuera
const doble2 = (n: number) => n * 2;
// Con efecto (no la uses en el render):
let contadorGlobal = 0;
const incrementar = () => { contadorGlobal++; };
```

---

4. Arrays: ordenar, filtrar, transformar

> Regla de oro: `sort`, `reverse`, `push`, `splice` **mutan**. Con el estado de React copia antes: `[...lista].sort(...)`.

4A. Ordenar

4.1 ★ Números ascendente

```
const asc = [...numeros].sort((a, b) => a - b);   // [1, 3, 9, 10, 25, 100]
```

4.2 ★ Números descendente

```
const desc = [...numeros].sort((a, b) => b - a);  // [100, 25, 10, 9, 3, 1]
```

4.3 ★★ La trampa: `sort()` sin comparador ordena como texto

```
[...numeros].sort();  // [1, 10, 100, 25, 3, 9]  ← mal para números
```

4.4 ★ Alfabético A→Z

```
const az = [...nombres].sort((a, b) => a.localeCompare(b, "es"));
```

4.5 ★ Alfabético Z→A

```
const za = [...nombres].sort((a, b) => b.localeCompare(a, "es"));
```

4.6 ★★ Alfabético ignorando mayúsculas y tildes

```
const base = [...nombres].sort((a, b) =>
  a.localeCompare(b, "es", { sensitivity: "base" })
);
```

4.7 ★★ Orden "natural" con números dentro del texto

```
const temas = ["Tema 10", "Tema 2", "Tema 1"];
temas.sort((a, b) => a.localeCompare(b, "es", { numeric: true })); // 1, 2, 10
```

4.8 ★ Productos por precio, barato primero

```
const baratos = [...productos].sort((a, b) => a.precio - b.precio);
```

4.9 ★ Productos por precio, caro primero

```
const caros = [...productos].sort((a, b) => b.precio - a.precio);
```

4.10 ★ Productos por nombre A→Z

```
const porNombre = [...productos].sort((a, b) => a.nombre.localeCompare(b.nombre, "es"));
```

4.11 ★ Productos por nombre Z→A

```
const porNombreDesc = [...productos].sort((a, b) => b.nombre.localeCompare(a.nombre, "es"));
```

4.12 ★★ Por nombre ignorando que "teclado" va en minúscula

```
[...productos].sort((a, b) => a.nombre.localeCompare(b.nombre, "es", { sensitivity: "base" }));
```

4.13 ★ Por categoría alfabética

```
const porCategoria = [...productos].sort((a, b) => a.categoria.localeCompare(b.categoria, "es"));
```

4.14 ★★ Por categoría y, dentro de cada una, por precio ascendente

```
[...productos].sort((a, b) =>
  a.categoria.localeCompare(b.categoria, "es") || a.precio - b.precio
);
```

4.15 ★★ Por categoría A→Z y precio descendente

```
[...productos].sort((a, b) =>
  a.categoria.localeCompare(b.categoria, "es") || b.precio - a.precio
);
```

4.16 ★★ Por categoría, luego precio, luego nombre (tres criterios)

```
[...productos].sort((a, b) =>
  a.categoria.localeCompare(b.categoria, "es") ||
  a.precio - b.precio ||
  a.nombre.localeCompare(b.nombre, "es")
);
```

4.17 ★★ Por categoría en un orden personalizado (no alfabético)

```
const ordenCat = ["Pantallas", "Periféricos", "Cables", "Muebles"];
[...productos].sort((a, b) => ordenCat.indexOf(a.categoria) - ordenCat.indexOf(b.categoria));
```

4.18 ★★ Por stock (más stock primero)

```
[...productos].sort((a, b) => b.stock - a.stock);
```

4.19 ★★ Sin stock al final

```
[...productos].sort((a, b) => Number(a.stock === 0) - Number(b.stock === 0));
```

4.20 ★★ Por fecha, más antiguo primero (texto ISO)

```
[...productos].sort((a, b) => a.fecha.localeCompare(b.fecha));
```

4.21 ★★ Por fecha, más reciente primero (con `Date`)

```
[...productos].sort((a, b) => new Date(b.fecha).getTime() - new Date(a.fecha).getTime());
```

4.22 ★★ Por longitud del nombre

```
[...productos].sort((a, b) => a.nombre.length - b.nombre.length);
```

4.23 ★ Candidatos por votos, el que más tiene primero

```
[...candidatos].sort((a, b) => b.votos - a.votos);
```

4.24 ★★ Candidatos por votos y, en empate, por nombre

```
[...candidatos].sort((a, b) => b.votos - a.votos || a.nombre.localeCompare(b.nombre, "es"));
```

4.25 ★ Candidatos por número de lista ascendente

```
[...candidatos].sort((a, b) => a.numeroLista - b.numeroLista);
```

4.26 ★★ Candidatos con lema primero

```
[...candidatos].sort((a, b) => Number(b.lema !== undefined) - Number(a.lema !== undefined));
```

4.27 ★★ Ordenar por un campo opcional (`lema`) sin que falle

```
[...candidatos].sort((a, b) => (a.lema ?? "").localeCompare(b.lema ?? "", "es"));
```

4.28 ★★ Tareas: pendientes primero, hechas al final

```
type Tarea = { id: number; texto: string; hecha: boolean };
const ordenarTareas = (t: Tarea[]) => [...t].sort((a, b) => Number(a.hecha) - Number(b.hecha));
```

4.29 ★★ Invertir un orden ya hecho (sin mutar)

```
const alReves = [...productos].sort((a, b) => a.precio - b.precio).reverse();
```

4.30 ★★★ `toSorted` (copia incluida, ES2023)

```
const ordenados = productos.toSorted((a, b) => a.precio - b.precio); // no muta
// Requiere "lib": ["ES2023"] en tsconfig; si no, usa [...lista].sort
```

4.31 ★★★ Función de ordenar por cualquier campo (genérica)

```
function ordenarPor<T, K extends keyof T>(lista: T[], campo: K, dir: "asc" | "desc" = "asc"): T[] {
  const f = dir === "asc" ? 1 : -1;
  return [...lista].sort((a, b) => (a[campo] < b[campo] ? -f : a[campo] > b[campo] ? f : 0));
}
ordenarPor(productos, "precio", "desc");
ordenarPor(candidatos, "nombre");
```

4.32 ★★★ Orden elegido con un `select` (tabla de comparadores)

```
type Orden = "votos-desc" | "votos-asc" | "nombre-asc" | "nombre-desc" | "lista-asc";

const comparadores: Record<Orden, (a: Candidato, b: Candidato) => number> = {
  "votos-desc":  (a, b) => b.votos - a.votos,
  "votos-asc":   (a, b) => a.votos - b.votos,
  "nombre-asc":  (a, b) => a.nombre.localeCompare(b.nombre, "es"),
  "nombre-desc": (a, b) => b.nombre.localeCompare(a.nombre, "es"),
  "lista-asc":   (a, b) => a.numeroLista - b.numeroLista,
};

const visibles = [...candidatos].sort(comparadores["votos-desc"]);
```

4.33 ★★★ Comparador reutilizable para texto

```
const porTexto = <T,>(sacar: (x: T) => string) => (a: T, b: T) =>
  sacar(a).localeCompare(sacar(b), "es", { sensitivity: "base" });

[...productos].sort(porTexto(p => p.nombre));
```

4.34 ★★★ Comparador reutilizable para números, con dirección

```
const porNumero = <T,>(sacar: (x: T) => number, dir: 1 | -1 = 1) => (a: T, b: T) =>
  (sacar(a) - sacar(b)) * dir;

[...productos].sort(porNumero(p => p.precio, -1)); // caro primero
```

4.35 ★★★ Encadenar varios comparadores

```
type Comp<T> = (a: T, b: T) => number;
const encadenar = <T,>(...comps: Comp<T>[]): Comp<T> => (a, b) => {
  for (const c of comps) {
    const r = c(a, b);
    if (r !== 0) return r;
  }
  return 0;
};

[...productos].sort(encadenar(
  porTexto<Producto>(p => p.categoria),
  porNumero<Producto>(p => p.precio, -1),
));
```

4.36 ★★ Mezclar un array al azar (sin mutar)

```
const barajar = <T,>(lista: T[]): T[] => {
  const copia = [...lista];
  for (let i = copia.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [copia[i], copia[j]] = [copia[j], copia[i]];
  }
  return copia;
};
```

4.37 ★★ Los 3 candidatos con más votos

```
const top3 = [...candidatos].sort((a, b) => b.votos - a.votos).slice(0, 3);
```

4.38 ★★ Los 3 productos más baratos

```
const tresBaratos = [...productos].sort((a, b) => a.precio - b.precio).slice(0, 3);
```

4.39 ★★ Ranking con posición (1.º, 2.º…)

```
const ranking = [...candidatos]
  .sort((a, b) => b.votos - a.votos)
  .map((c, i) => ({ ...c, posicion: i + 1 }));
```

4.40 ★★★ Ranking con empates (misma posición si mismos votos)

```
const ordenados = [...candidatos].sort((a, b) => b.votos - a.votos);
const conPuesto = ordenados.map((c, i) => ({
  ...c,
  puesto: ordenados.findIndex(x => x.votos === c.votos) + 1,
}));
```

4B. Filtrar y buscar

4.41 ★ Productos de una categoría

```
productos.filter(p => p.categoria === "Muebles");
```

4.42 ★ Productos baratos (menos de 50 €)

```
productos.filter(p => p.precio < 50);
```

4.43 ★★ Rango de precios

```
const entre = (min: number, max: number) => productos.filter(p => p.precio >= min && p.precio <= max);
```

4.44 ★ Buscar por texto (sin distinguir mayúsculas)

```
const buscar = (texto: string) =>
  productos.filter(p => p.nombre.toLowerCase().includes(texto.toLowerCase()));
```

4.45 ★★ Buscar sin distinguir tildes

```
const sinTildes = (s: string) => s.normalize("NFD").replace(/\p{Diacritic}/gu, "").toLowerCase();
candidatos.filter(c => sinTildes(c.nombre).includes(sinTildes("alvaro"))); // encuentra Álvaro
```

4.46 ★ Candidatos sin votos

```
candidatos.filter(c => c.votos === 0);
```

4.47 ★★ Candidatos líderes (puede haber empate)

```
const max = Math.max(0, ...candidatos.map(c => c.votos));
const lideres = candidatos.filter(c => c.votos > 0 && c.votos === max);
```

4.48 ★ Candidatos con lema

```
candidatos.filter(c => c.lema !== undefined);
```

4.49 ★★ Filtro con varias condiciones combinadas

```
productos.filter(p => p.categoria === "Periféricos" && p.stock > 0 && p.precio <= 40);
```

4.50 ★★ Excluir un id (borrar)

```
productos.filter(p => p.id !== 3);
```

4.51 ★ Primer elemento que cumple (`find`)

```
const monitor = productos.find(p => p.nombre === "Monitor"); // Producto | undefined
```

4.52 ★★ Posición de un elemento (`findIndex`)

```
const pos = productos.findIndex(p => p.id === 5); // -1 si no existe
```

4.53 ★ ¿Hay alguno que cumpla? (`some`)

```
const hayAgotados = productos.some(p => p.stock === 0);
```

4.54 ★ ¿Cumplen todos? (`every`)

```
const todosCaros = productos.every(p => p.precio > 5);
```

4.55 ★ ¿Contiene un valor? (`includes`)

```
const hayCable = productos.map(p => p.nombre).includes("Cable HDMI");
```

4.56 ★★ Filtrar y ordenar en la misma cadena

```
productos
  .filter(p => p.stock > 0)
  .sort((a, b) => a.precio - b.precio);
// filter ya devuelve copia, así que aquí sort no muta el original
```

4.57 ★★ Filtrar por un conjunto de categorías

```
const permitidas = ["Cables", "Pantallas"];
productos.filter(p => permitidas.includes(p.categoria));
```

4.58 ★★ Filtrar por estado usando `obtenerEstado`

```
const max2 = Math.max(0, ...candidatos.map(c => c.votos));
candidatos.filter(c => obtenerEstado(c.votos, max2) === "algunos");
```

4C. Transformar y calcular

4.59 ★ Sacar solo los nombres

```
const soloNombres: string[] = productos.map(p => p.nombre);
```

4.60 ★★ Crear objetos nuevos con otra forma

```
const resumen = productos.map(p => ({ id: p.id, etiqueta: `${p.nombre} (${p.precio} €)` }));
```

4.61 ★★ Subir un 10 % todos los precios (sin mutar)

```
const subidos = productos.map(p => ({ ...p, precio: Math.round(p.precio * 1.1 * 100) / 100 }));
```

4.62 ★ Suma de precios

```
const sumaPrecios = productos.reduce((s, p) => s + p.precio, 0);
```

4.63 ★ Valor total del inventario

```
const inventario = productos.reduce((s, p) => s + p.precio * p.stock, 0);
```

4.64 ★★ Precio medio

```
const media = productos.length === 0 ? 0 : sumaPrecios / productos.length;
```

4.65 ★★ Producto más caro con `reduce`

```
const masCaro = productos.reduce<Producto | null>(
  (mejor, p) => (mejor === null || p.precio > mejor.precio ? p : mejor),
  null
);
```

4.66 ★★ Producto más barato

```
const masBarato = [...productos].sort((a, b) => a.precio - b.precio)[0];
```

4.67 ★★ Contar productos por categoría

```
const conteo = productos.reduce<Record<string, number>>((acc, p) => {
  acc[p.categoria] = (acc[p.categoria] ?? 0) + 1;
  return acc;
}, {});
// { Periféricos: 3, Pantallas: 1, Cables: 1, Muebles: 2 }
```

4.68 ★★★ Agrupar productos por categoría

```
const grupos = productos.reduce<Record<string, Producto[]>>((acc, p) => {
  (acc[p.categoria] ??= []).push(p);
  return acc;
}, {});
```

4.69 ★★★ Sumar precio por categoría

```
const sumaPorCat = productos.reduce<Record<string, number>>((acc, p) => {
  acc[p.categoria] = (acc[p.categoria] ?? 0) + p.precio;
  return acc;
}, {});
```

4.70 ★★ Categorías únicas con `Set`

```
const categorias: string[] = [...new Set(productos.map(p => p.categoria))];
```

4.71 ★★ Categorías únicas y ordenadas

```
const categoriasOrd = [...new Set(productos.map(p => p.categoria))].sort((a, b) => a.localeCompare(b,
    "es"));
```

4.72 ★★ Recorrer un objeto con `Object.entries`

```
for (const [cat, cantidad] of Object.entries(conteo)) {
  console.log(`${cat}: ${cantidad}`);
}
```

4.73 ★★★ Convertir `entries` otra vez en objeto

```
const soloMuchos = Object.fromEntries(Object.entries(conteo).filter(([, n]) => n > 1));
```

4.74 ★★ Paginar (página 2, de 3 en 3)

```
const pagina = (lista: Producto[], n: number, tam: number) => lista.slice((n - 1) * tam, n * tam);
pagina(productos, 2, 3);
```

4.75 ★★ Número de páginas

```
const totalPaginas = Math.ceil(productos.length / 3);
```

4.76 ★★★ Dividir en trozos (chunk)

```
const trozos = <T,>(lista: T[], tam: number): T[][] =>
  Array.from({ length: Math.ceil(lista.length / tam) }, (_, i) => lista.slice(i * tam, i * tam + tam));
```

4.77 ★★ Unir nombres en un texto

```
productos.map(p => p.nombre).join(", ");
```

4.78 ★★ Aplanar con `flatMap`

```
const frases = ["hola mundo", "buen día"];
const palabras = frases.flatMap(f => f.split(" ")); // ["hola","mundo","buen","día"]
```

4.79 ★ Concatenar dos listas

```
const todos = [...productos, ...productos.slice(0, 1)];
```

4.80 ★★ Eliminar duplicados por `id`

```
const unicos = [...new Map(productos.map(p => [p.id, p])).values()];
```

4.81 ★★ Reemplazar un elemento por id

```
const reemplazado = productos.map(p => (p.id === 2 ? { ...p, precio: 35 } : p));
```

4.82 ★★ Insertar al principio en lugar de al final

```
const conNuevo = [{ ...productos[0], id: 99 }, ...productos];
```

4.83 ★★ Mover un elemento al final

```
const moverAlFinal = (lista: Producto[], id: number) => [
  ...lista.filter(p => p.id !== id),
  ...lista.filter(p => p.id === id),
];
```

4.84 ★★★ Intercambiar dos posiciones sin mutar

```
const intercambiar = <T,>(lista: T[], i: number, j: number): T[] => {
  const copia = [...lista];
  [copia[i], copia[j]] = [copia[j], copia[i]];
  return copia;
};
```

4.85 ★★ Crear un array de 1 a N

```
const del1al5 = Array.from({ length: 5 }, (_, i) => i + 1); // [1,2,3,4,5]
```

4.86 ★★ Votos de cada candidato como array de números

```
const votos: number[] = candidatos.map(c => c.votos);
```

4.87 ★★ Candidatos con su porcentaje añadido

```
const total = candidatos.reduce((s, c) => s + c.votos, 0);
const conPorc = candidatos.map(c => ({ ...c, porcentaje: total === 0 ? 0 : Math.round((c.votos / total) *
    100) }));
```

---

5. Genéricos y utility types

5.1 ★★ Función genérica que devuelve el primer elemento

```
function primero<T>(lista: T[]): T | undefined {
  return lista[0];
}
primero([1, 2, 3]);     // number | undefined
primero(["a", "b"]);    // string | undefined
```

5.2 ★★ Último elemento

```
const ultimo = <T,>(lista: T[]): T | undefined => lista[lista.length - 1];
```

5.3 ★★ Genérico con restricción (`extends`)

```
function conId<T extends { id: number }>(lista: T[], id: number): T | undefined {
  return lista.find(x => x.id === id);
}
conId(productos, 3);
conId(candidatos, 1);
```

5.4 ★★★ Quitar un elemento por id (para cualquier lista con `id`)

```
const quitar = <T extends { id: number }>(lista: T[], id: number): T[] => lista.filter(x => x.id !== id);
```

5.5 ★★★ Modificar un elemento por id con cambios parciales

```
function actualizar<T extends { id: number }>(lista: T[], id: number, cambios: Partial<T>): T[] {
  return lista.map(x => (x.id === id ? { ...x, ...cambios } : x));
}
actualizar(candidatos, 2, { lema: "Más deporte" });
```

5.6 ★★ `Partial`: todo opcional

```
type CambiosProducto = Partial<Producto>;
const c: CambiosProducto = { precio: 20 };
```

5.7 ★★ `Omit`: todo menos algunas claves (producto nuevo sin id)

```
type ProductoNuevo = Omit<Producto, "id">;
```

5.8 ★★ `Pick`: solo algunas claves

```
type ProductoResumen = Pick<Producto, "nombre" | "precio">;
```

5.9 ★★ `Required`: todo obligatorio

```
type CandidatoCompleto = Required<Candidato>; // lema pasa a ser obligatorio
```

5.10 ★★ `Readonly`: no se puede cambiar

```
const fijo: Readonly<Candidato> = candidatos[0];
// fijo.votos = 10; // Error
```

5.11 ★★ `Record` con claves de una unión

```
type Color = "ninguno" | "algunos" | "lider";
const colores: Record<Estado, string> = { ninguno: "gray", algunos: "orange", lider: "gold" };
```

5.12 ★★★ `ReturnType` y `Parameters`

```
type R = ReturnType<typeof obtenerEstado>;   // Estado
type P = Parameters<typeof obtenerEstado>;   // [votos: number, maximo: number]
```

5.13 ★★★ `Exclude` y `Extract`

```
type SinLider = Exclude<Estado, "lider">;     // "ninguno" | "algunos"
type SoloLider = Extract<Estado, "lider">;    // "lider"
```

5.14 ★★★ `NonNullable`

```
type Seguro = NonNullable<string | null | undefined>; // string
```

5.15 ★★★ `Awaited` (resultado de una promesa)

```
async function cargar(): Promise<Producto[]> { return productos; }
type Datos = Awaited<ReturnType<typeof cargar>>; // Producto[]
```

5.16 ★★ Interfaz genérica para respuestas

```
interface Respuesta<T> { ok: boolean; datos: T }
const r: Respuesta<Producto[]> = { ok: true, datos: productos };
```

5.17 ★★★ Genérico con dos tipos

```
function emparejar<A, B>(a: A, b: B): [A, B] {
  return [a, b];
}
emparejar("votos", 7); // [string, number]
```

5.18 ★★★ Tipo mapeado propio

```
type Opcionales<T> = { [K in keyof T]?: T[K] };
```

5.19 ★★★ Claves cuyo valor es un número

```
type ClavesNumero<T> = { [K in keyof T]: T[K] extends number ? K : never }[keyof T];
type CamposNumericos = ClavesNumero<Producto>; // "id" | "precio" | "stock"
```

5.20 ★★★ Sumar cualquier campo numérico de una lista

```
function sumarCampo<T>(lista: T[], campo: ClavesNumero<T>): number {
  return lista.reduce((s, x) => s + (x[campo] as number), 0);
}
sumarCampo(productos, "stock");
sumarCampo(candidatos, "votos");
```

5.21 ★★ `useState` con genérico

```
const [lista, setLista] = useState<Candidato[]>([]);
const [elegido, setElegido] = useState<Candidato | null>(null);
```

5.22 ★★ Tipo de un valor opcional en un estado de formulario

```
type Formulario = { nombre: string; lista: string; lema: string };
const [datos, setDatos] = useState<Formulario>({ nombre: "", lista: "", lema: "" });
```

---

6. React: componentes, props y renderizado

6A. Props

6.1 ★ Componente sin props

```
function Titulo() {
  return <h1>Elección de delegado</h1>;
}
```

6.2 ★ Props con tipo `type`

```
type SaludoProps = { nombre: string };
function Saludo({ nombre }: SaludoProps) {
  return <p>Hola, {nombre}</p>;
}
```

6.3 ★ Prop opcional con valor por defecto (`textoBoton`)

```
type BotonProps = { texto?: string };
function Boton({ texto = "Votar" }: BotonProps) {
  return <button>{texto}</button>;
}
```

6.4 ★ Prop numérica y booleana

```
<Producto precio={29.99} disponible={true} nombre="Teclado" />
```

6.5 ★ Prop que es un objeto

```
type TarjetaProps = { candidato: Candidato };
function Tarjeta({ candidato }: TarjetaProps) {
  return <h2>{candidato.nombre}</h2>;
}
```

6.6 ★★ Prop que es una función (`alVotar`)

```
type Props = { candidato: Candidato; alVotar: (id: number) => void };
function Tarjeta2({ candidato, alVotar }: Props) {
  return <button onClick={() => alVotar(candidato.id)}>Votar</button>;
}
```

6.7 ★★ Prop que es una unión de literales

```
type AvisoProps = { tipo: "info" | "aviso" | "error"; texto: string };
function Aviso({ tipo, texto }: AvisoProps) {
  return <p className={`aviso aviso--${tipo}`}>{texto}</p>;
}
```

6.8 ★★ `children`

```
import type { ReactNode } from "react";
type CuadroProps = { titulo: string; children: ReactNode };
function Cuadro({ titulo, children }: CuadroProps) {
  return <section><h2>{titulo}</h2>{children}</section>;
}
```

6.9 ★★ `children` opcional

```
type ContenedorProps = { children?: ReactNode };
```

6.10 ★★ Props con un array

```
type ListaProps = { candidatos: Candidato[] };
function Lista({ candidatos }: ListaProps) {
  return <ul>{candidatos.map(c => <li key={c.id}>{c.nombre}</li>)}</ul>;
}
```

6.11 ★★ Reutilizar un tipo en las props con `Pick`

```
type FilaProps = Pick<Candidato, "nombre" | "votos">;
function Fila({ nombre, votos }: FilaProps) {
  return <tr><td>{nombre}</td><td>{votos}</td></tr>;
}
```

6.12 ★★ Pasar todas las propiedades con spread

```
<Fila {...candidatos[0]} />
```

6.13 ★★★ Props extendiendo las de un elemento HTML

```
import type { ComponentProps } from "react";
type BotonPropio = ComponentProps<"button"> & { variante?: "primario" | "peligro" };
function BotonX({ variante = "primario", ...resto }: BotonPropio) {
  return <button className={`btn btn--${variante}`} {...resto} />;
}
```

6.14 ★★ Prop de estilo en línea

```
<p style={{ color: "red", fontSize: "20px" }}>Atención</p>
```

6.15 ★★ Clase CSS según el estado (`tarjeta--lider`)

```
<article className={`tarjeta tarjeta--${estado}`}>…</article>
```

6.16 ★★ Clases condicionales

```
<li className={t.hecha ? "tarea tarea--hecha" : "tarea"}>{t.texto}</li>
```

6B. Renderizado condicional

6.17 ★ Ternario

```
<p>{estado === "lider" ? "♛ Va en cabeza" : "Sigue luchando"}</p>
```

6.18 ★ `&&` para mostrar algo solo si se cumple

```
{estado === "ninguno" && <p>Todavía sin votos</p>}
```

6.19 ★ Mostrar el lema solo si existe

```
{candidato.lema && <p className="lema">«{candidato.lema}»</p>}
```

6.20 ★★ La trampa del `0` con `&&`

```
{lista.length && <Lista />}        // pinta un 0 si está vacía
{lista.length > 0 && <Lista />}    // correcto
```

6.21 ★ Mensaje cuando la lista está vacía

```
{candidatos.length === 0
  ? <p className="vacio">Todavía no hay candidaturas.</p>
  : <ul>{candidatos.map(c => <li key={c.id}>{c.nombre}</li>)}</ul>}
```

6.22 ★★ Retorno temprano

```
function Panel({ usuario }: { usuario: string | null }) {
  if (!usuario) return <p>Inicia sesión</p>;
  return <p>Hola, {usuario}</p>;
}
```

6.23 ★★ Tres casos con una tabla en vez de `if` anidados

```
const mensajes: Record<Estado, string | null> = {
  ninguno: "Todavía sin votos",
  algunos: null,
  lider: "♛ Va en cabeza",
};
// …
{mensajes[estado] && <p>{mensajes[estado]}</p>}
```

6.24 ★★ Mostrar el singular o el plural

```
<p>{votos} {votos === 1 ? "voto" : "votos"}</p>
```

6.25 ★★ Botón desactivado

```
<button disabled={votos >= 10}>Votar</button>
```

6C. Listas

6.26 ★ Lista con `map` y `key`

```
<ul>{productos.map(p => <li key={p.id}>{p.nombre}</li>)}</ul>
```

6.27 ★ Componente en una lista con `key` en el componente

```
{candidatos.map(c => (
  <TarjetaCandidato key={c.id} candidato={c} estado={obtenerEstado(c.votos, maximo)} alVotar={votar} />
))}
```

6.28 ★★ Lista ordenada por precio

```
<ul>
  {[...productos].sort((a, b) => a.precio - b.precio).map(p => (
    <li key={p.id}>{p.nombre} — {p.precio} €</li>
  ))}
</ul>
```

6.29 ★★ Lista agrupada por categoría

```
{Object.entries(grupos).map(([categoria, items]) => (
  <section key={categoria}>
    <h3>{categoria}</h3>
    <ul>{items.map(p => <li key={p.id}>{p.nombre}</li>)}</ul>
  </section>
))}
```

6.30 ★★ Tabla con cabecera y filas

```
<table>
  <thead><tr><th>Nombre</th><th>Precio</th></tr></thead>
  <tbody>
    {productos.map(p => <tr key={p.id}><td>{p.nombre}</td><td>{p.precio} €</td></tr>)}
  </tbody>
</table>
```

6.31 ★★ Filas alternas (par/impar)

```
{productos.map((p, i) => <li key={p.id} className={i % 2 === 0 ? "par" : "impar"}>{p.nombre}</li>)}
```

6.32 ★★ Mostrar solo los N primeros y «ver más»

```
const [verTodos, setVerTodos] = useState(false);
const mostrados = verTodos ? productos : productos.slice(0, 3);
// …
<button onClick={() => setVerTodos(v => !v)}>{verTodos ? "Ver menos" : "Ver más"}</button>
```

6.33 ★★ Fragmento para devolver dos elementos

```
return (
  <>
    <h2>Título</h2>
    <p>Texto</p>
  </>
);
```

6.34 ★★ Una `key` incorrecta: el índice (cuándo se nota)

```
{lista.map((x, i) => <li key={i}>{x}</li>)}   // mal si la lista se reordena o se borra
```

---

7. Estado con `useState`

7A. Valores simples

7.1 ★ Contador

```
const [contador, setContador] = useState(0);
<button onClick={() => setContador(prev => prev + 1)}>+</button>
```

7.2 ★ Contador que no baja de 0

```
setContador(prev => Math.max(0, prev - 1));
```

7.3 ★★ Contador con límite superior e inferior

```
setContador(prev => Math.min(10, prev + 1));
setContador(prev => Math.max(0, prev - 1));
```

7.4 ★ Reiniciar

```
<button onClick={() => setContador(0)}>Reiniciar</button>
```

7.5 ★★ Sumar de 1 en 1 tres veces en un solo clic

```
setContador(p => p + 1);
setContador(p => p + 1);
setContador(p => p + 1);   // sube 3 (con contador + 1 subiría 1)
```

7.6 ★ Interruptor (booleano)

```
const [encendido, setEncendido] = useState(false);
<button onClick={() => setEncendido(prev => !prev)}>{encendido ? "Apagar" : "Encender"}</button>
```

7.7 ★ Texto controlado

```
const [texto, setTexto] = useState("");
<input value={texto} onChange={e => setTexto(e.target.value)} />
```

7.8 ★ Contador de caracteres derivado

```
const caracteres = texto.length;   // no necesita otro useState
```

7.9 ★★ Límite de caracteres con aviso

```
const MAX = 60;
const quedan = MAX - texto.length;
<p className={quedan < 0 ? "error" : ""}>{quedan} caracteres restantes</p>
```

7.10 ★★ Número que viene de un `<input type="number">`

```
const [cantidad, setCantidad] = useState(1);
<input type="number" value={cantidad} onChange={e => setCantidad(Number(e.target.value))} />
```

7.11 ★★ Estado que puede ser `null` (elegido)

```
const [elegido, setElegido] = useState<number | null>(null);
<li onClick={() => setElegido(c.id)} className={elegido === c.id ? "activo" : ""}>{c.nombre}</li>
```

7.12 ★★ Alternar la selección (clic otra vez para quitar)

```
setElegido(prev => (prev === c.id ? null : c.id));
```

7.13 ★★ Estado con una unión de literales (pestañas)

```
type Pestana = "resumen" | "detalle" | "ajustes";
const [pestana, setPestana] = useState<Pestana>("resumen");
```

7.14 ★★ Estado para el criterio de orden

```
const [orden, setOrden] = useState<Orden>("votos-desc");
const visibles = [...candidatos].sort(comparadores[orden]);
```

7.15 ★★ Estado para el criterio y la dirección por separado

```
const [campo, setCampo] = useState<"nombre" | "precio">("nombre");
const [asc, setAsc] = useState(true);
const lista = [...productos].sort((a, b) => {
  const r = campo === "nombre" ? a.nombre.localeCompare(b.nombre, "es") : a.precio - b.precio;
  return asc ? r : -r;
});
```

7B. Objetos

7.16 ★★ Cambiar un campo de un objeto de estado

```
setDatos(prev => ({ ...prev, nombre: "Luis" }));
```

7.17 ★★ Cambiar un campo usando su `name`

```
setDatos(prev => ({ ...prev, [e.target.name]: e.target.value }));
```

7.18 ★★ Objeto anidado

```
setPerfil(prev => ({ ...prev, direccion: { ...prev.direccion, ciudad: "Oviedo" } }));
```

7.19 ★ Vaciar un objeto de estado

```
setDatos({ nombre: "", lista: "", lema: "" });
```

7.20 ★★ Quitar una propiedad opcional (el lema)

```
setCandidato(prev => ({ ...prev, lema: undefined }));
```

7C. Arrays: CRUD completo

7.21 ★ Añadir al final

```
setCandidatos(prev => [...prev, nuevo]);
```

7.22 ★ Añadir al principio

```
setCandidatos(prev => [nuevo, ...prev]);
```

7.23 ★ Votar (apartado 4 del ensayo)

```
const votar = (id: number) =>
  setCandidatos(prev => prev.map(c => (c.id === id ? { ...c, votos: c.votos + 1 } : c)));
```

7.24 ★★ Quitar un voto sin bajar de 0

```
const quitarVoto = (id: number) =>
  setCandidatos(prev => prev.map(c => (c.id === id ? { ...c, votos: Math.max(0, c.votos - 1) } : c)));
```

7.25 ★★ Votar de 5 en 5

```
const votarCinco = (id: number) =>
  setCandidatos(prev => prev.map(c => (c.id === id ? { ...c, votos: c.votos + 5 } : c)));
```

7.26 ★★ Limitar los votos máximos por candidato

```
const LIMITE = 20;
const votarLimitado = (id: number) =>
  setCandidatos(prev => prev.map(c => (c.id === id && c.votos < LIMITE ? { ...c, votos: c.votos + 1 } :
      c)));
```

7.27 ★ Eliminar un candidato

```
const eliminar = (id: number) => setCandidatos(prev => prev.filter(c => c.id !== id));
```

7.28 ★★ Poner a cero los votos de todos

```
const reiniciar = () => setCandidatos(prev => prev.map(c => ({ ...c, votos: 0 })));
```

7.29 ★★ Poner a cero los votos de uno

```
const reiniciarUno = (id: number) =>
  setCandidatos(prev => prev.map(c => (c.id === id ? { ...c, votos: 0 } : c)));
```

7.30 ★★ Editar el lema de un candidato

```
const cambiarLema = (id: number, lema: string) =>
  setCandidatos(prev => prev.map(c => (c.id === id ? { ...c, lema } : c)));
```

7.31 ★★ Alternar `hecha` en una tarea

```
setTareas(prev => prev.map(t => (t.id === id ? { ...t, hecha: !t.hecha } : t)));
```

7.32 ★★ Marcar todas como hechas

```
setTareas(prev => prev.map(t => ({ ...t, hecha: true })));
```

7.33 ★★ Borrar solo las hechas

```
setTareas(prev => prev.filter(t => !t.hecha));
```

7.34 ★★ Añadir al carrito o sumar cantidad si ya está

```
const agregar = (p: Producto) =>
  setCarrito(prev =>
    prev.some(i => i.id === p.id)
      ? prev.map(i => (i.id === p.id ? { ...i, cantidad: i.cantidad + 1 } : i))
      : [...prev, { ...p, cantidad: 1 }]
  );
```

7.35 ★★ Restar cantidad y quitar la línea si llega a 0

```
const restar = (id: number) =>
  setCarrito(prev =>
    prev
      .map(i => (i.id === id ? { ...i, cantidad: i.cantidad - 1 } : i))
      .filter(i => i.cantidad > 0)
  );
```

7.36 ★★ Vaciar la lista

```
setCarrito([]);
```

7.37 ★★ Evitar duplicados al añadir (mismo número de lista)

```
const anadir = (nuevo: Candidato) =>
  setCandidatos(prev =>
    prev.some(c => c.numeroLista === nuevo.numeroLista) ? prev : [...prev, nuevo]
  );
```

7.38 ★★★ Ordenar el estado y guardarlo ordenado (botón «ordenar»)

```
const ordenarPorVotos = () => setCandidatos(prev => [...prev].sort((a, b) => b.votos - a.votos));
```

7.39 ★★ Ordenar solo al mostrar (mejor: es dato derivado)

```
const visibles = [...candidatos].sort((a, b) => b.votos - a.votos);
```

7.40 ★★★ Subir un elemento una posición

```
const subir = (i: number) =>
  setLista(prev => {
    if (i === 0) return prev;
    const copia = [...prev];
    [copia[i - 1], copia[i]] = [copia[i], copia[i - 1]];
    return copia;
  });
```

7.41 ★★ Deshacer el último elemento añadido

```
setLista(prev => prev.slice(0, -1));
```

7.42 ★★ Estado inicial calculado una sola vez (inicialización perezosa)

```
const [lista, setLista] = useState<Candidato[]>(() => candidatosIniciales.slice());
```

7D. Datos derivados (lo que NO va en el estado)

7.43 ★ Total de votos

```
const totalVotos = candidatos.reduce((s, c) => s + c.votos, 0);
```

7.44 ★ Máximo de votos (0 si no hay candidatos)

```
const maximo = Math.max(0, ...candidatos.map(c => c.votos));
```

7.45 ★ Pendientes

```
const pendientes = tareas.filter(t => !t.hecha).length;
```

7.46 ★★ Total del carrito

```
const totalCarrito = carrito.reduce((s, i) => s + i.precio * i.cantidad, 0);
```

7.47 ★★ Número de unidades en el carrito

```
const unidades = carrito.reduce((s, i) => s + i.cantidad, 0);
```

7.48 ★★ ¿Está vacío? ¿Hay alguno?

```
const hayCandidatos = candidatos.length > 0;
```

7.49 ★★ Porcentaje de tareas hechas

```
const progreso = tareas.length === 0 ? 0 : Math.round((tareas.filter(t => t.hecha).length / tareas.length) *
    100);
```

7.50 ★★ Lista visible = filtrada + ordenada (todo derivado)

```
const visibles = candidatos
  .filter(c => c.nombre.toLowerCase().includes(busqueda.toLowerCase()))
  .sort(comparadores[orden]);   // filter ya devolvió una copia nueva
```

---

8. Eventos y formularios

8A. Eventos

8.1 ★ Pasar la función, no ejecutarla

```
<button onClick={sumar}>Sumar</button>          // correcto
<button onClick={sumar()}>Sumar</button>        // MAL: se ejecuta al dibujar
```

8.2 ★ Pasar un argumento con función flecha

```
<button onClick={() => alVotar(candidato.id)}>Votar</button>
```

8.3 ★★ Manejador extraído y tipado (clic)

```
import type { MouseEvent } from "react";
const clic = (e: MouseEvent<HTMLButtonElement>) => console.log(e.currentTarget.textContent);
```

8.4 ★★ Manejador tipado de `input`

```
import type { ChangeEvent } from "react";
const cambiar = (e: ChangeEvent<HTMLInputElement>) => setTexto(e.target.value);
```

8.5 ★★ Manejador tipado de `select`

```
const cambiarOrden = (e: ChangeEvent<HTMLSelectElement>) => setOrden(e.target.value as Orden);
```

8.6 ★★ Manejador tipado de `textarea`

```
const cambiarLema = (e: ChangeEvent<HTMLTextAreaElement>) => setLema(e.target.value);
```

8.7 ★★ Tecla pulsada (Enter para añadir)

```
import type { KeyboardEvent } from "react";
const tecla = (e: KeyboardEvent<HTMLInputElement>) => {
  if (e.key === "Enter") anadir();
};
```

8.8 ★★ Tecla Escape para vaciar

```
const tecla2 = (e: KeyboardEvent<HTMLInputElement>) => {
  if (e.key === "Escape") setTexto("");
};
```

8.9 ★★ `preventDefault` en el envío

```
import type { FormEvent } from "react";
function manejarEnvio(evento: FormEvent<HTMLFormElement>) {
  evento.preventDefault();
  // …
}
```

8.10 ★★ `stopPropagation` (botón dentro de una tarjeta clicable)

```
<article onClick={() => seleccionar(c.id)}>
  <button onClick={(e) => { e.stopPropagation(); votar(c.id); }}>Votar</button>
</article>
```

8.11 ★★ `target` frente a `currentTarget`

```
<button onClick={(e) => console.log(e.target, e.currentTarget)}>
  <span>Votar</span>   {/* si pulsas el span: target = span, currentTarget = button */}
</button>
```

8.12 ★★ Evento de ratón al pasar por encima

```
<li onMouseEnter={() => setHover(c.id)} onMouseLeave={() => setHover(null)}>{c.nombre}</li>
```

8.13 ★★ Enfocar al hacer clic con `useRef`

```
const ref = useRef<HTMLInputElement>(null);
<button onClick={() => ref.current?.focus()}>Enfocar</button>
<input ref={ref} />
```

8B. Formularios

8.14 ★ Input controlado

```
const [nombre, setNombre] = useState("");
<input value={nombre} onChange={e => setNombre(e.target.value)} />
```

8.15 ★ `label` enlazada con `htmlFor`

```
<label htmlFor="nombre">Nombre</label>
<input id="nombre" value={nombre} onChange={e => setNombre(e.target.value)} />
```

8.16 ★ Los tres campos del ensayo (nombre, número, lema)

```
const [nombre, setNombre] = useState("");
const [numero, setNumero] = useState("");   // texto: así lo entrega el input
const [lema, setLema] = useState("");
```

8.17 ★ Validación sin estado extra

```
const valido =
  nombre.trim() !== "" &&
  numero !== "" &&
  Number(numero) >= 1 &&
  Number(numero) <= 30;
```

8.18 ★ Botón deshabilitado según la validación

```
<button type="submit" disabled={!valido}>Presentar candidatura</button>
```

8.19 ★★ Envío completo del ensayo (apartado 5)

```
function manejarEnvio(evento: FormEvent<HTMLFormElement>) {
  evento.preventDefault();
  const limpioLema = lema.trim();
  const nuevo: Candidato = {
    id: Date.now(),
    nombre: nombre.trim(),
    numeroLista: Number(numero),
    votos: 0,
    ...(limpioLema !== "" && { lema: limpioLema }),
  };
  alAnadir(nuevo);
  setNombre("");
  setNumero("");
  setLema("");
}
```

8.20 ★★ Otra forma de añadir el lema solo si existe

```
const nuevo2: Candidato = { id: Date.now(), nombre, numeroLista: Number(numero), votos: 0 };
if (lema.trim() !== "") nuevo2.lema = lema.trim();
```

8.21 ★★ Un solo estado para todo el formulario

```
type Datos = { nombre: string; numero: string; lema: string };
const vacio: Datos = { nombre: "", numero: "", lema: "" };
const [datos, setDatos] = useState<Datos>(vacio);

const cambiar = (e: ChangeEvent<HTMLInputElement>) => {
  const { name, value } = e.target;
  setDatos(prev => ({ ...prev, [name]: value }));
};
// al enviar: setDatos(vacio);
```

8.22 ★★ Checkbox

```
const [acepta, setAcepta] = useState(false);
<input type="checkbox" checked={acepta} onChange={e => setAcepta(e.target.checked)} />
```

8.23 ★★ Select controlado

```
<select value={categoria} onChange={e => setCategoria(e.target.value)}>
  <option value="">Todas</option>
  {categorias.map(c => <option key={c} value={c}>{c}</option>)}
</select>
```

8.24 ★★ Select para elegir el orden

```
<select value={orden} onChange={e => setOrden(e.target.value as Orden)}>
  <option value="votos-desc">Más votados</option>
  <option value="votos-asc">Menos votados</option>
  <option value="nombre-asc">Nombre A–Z</option>
  <option value="nombre-desc">Nombre Z–A</option>
  <option value="lista-asc">Número de lista</option>
</select>
```

8.25 ★★ Radio buttons

```
{(["asc", "desc"] as const).map(d => (
  <label key={d}>
    <input type="radio" name="dir" value={d} checked={dir === d} onChange={() => setDir(d)} />
    {d === "asc" ? "Ascendente" : "Descendente"}
  </label>
))}
```

8.26 ★★ Validación con mensajes de error por campo

```
type Errores = { nombre?: string; numero?: string };
const validar = (d: Datos): Errores => {
  const e: Errores = {};
  if (!d.nombre.trim()) e.nombre = "El nombre es obligatorio";
  const n = Number(d.numero);
  if (d.numero === "" || !Number.isInteger(n) || n < 1 || n > 30) e.numero = "Entre 1 y 30";
  return e;
};
const errores = validar(datos);                 // derivado
const hayErrores = Object.keys(errores).length > 0;
```

8.27 ★★ Mostrar el error solo si el campo se ha tocado

```
const [tocado, setTocado] = useState(false);
<input value={nombre} onChange={e => setNombre(e.target.value)} onBlur={() => setTocado(true)} />
{tocado && !nombre.trim() && <p className="error">El nombre es obligatorio</p>}
```

8.28 ★★ Rechazar un número de lista repetido

```
const repetido = candidatos.some(c => c.numeroLista === Number(numero));
// y en el botón: disabled={!valido || repetido}
```

8.29 ★★ Limitar la longitud del lema

```
<input maxLength={60} value={lema} onChange={e => setLema(e.target.value)} />
```

8.30 ★★ Buscador que filtra mientras escribes

```
const [busqueda, setBusqueda] = useState("");
const filtrados = candidatos.filter(c => c.nombre.toLowerCase().includes(busqueda.trim().toLowerCase()));
```

8.31 ★★ Botón para limpiar la búsqueda

```
{busqueda && <button type="button" onClick={() => setBusqueda("")}>×</button>}
```

8.32 ★★ Convertir el valor de un input a número con seguridad

```
const n = Number(e.target.value);
if (!Number.isNaN(n)) setCantidad(n);
```

8.33 ★★ Convertir con `parseInt` y base

```
const entero = parseInt(numero, 10);   // NaN si no es número
```

---

9. Elevar el estado, `useEffect` y datos externos

9.1 ★ El padre guarda, el hijo recibe valor y callback

```
type BusquedaProps = { valor: string; alCambiar: (v: string) => void };
function Busqueda({ valor, alCambiar }: BusquedaProps) {
  return <input value={valor} onChange={e => alCambiar(e.target.value)} />;
}
// En App: <Busqueda valor={filtro} alCambiar={setFiltro} />
```

9.2 ★★ El hijo avisa al padre con un objeto

```
<FormularioCandidato alAnadir={(c) => setCandidatos(prev => [...prev, c])} />
```

9.3 ★★ Conectar con App (apartado 5): función con nombre

```
const anadirCandidato = (nuevo: Candidato) => setCandidatos(prev => [...prev, nuevo]);
<FormularioCandidato alAnadir={anadirCandidato} />
```

9.4 ★★ Dos hermanos que comparten un dato

```
// App guarda "orden" y lo pasa a Selector (para cambiar) y a Lista (para leer)
<Selector orden={orden} alCambiar={setOrden} />
<Lista candidatos={candidatos} orden={orden} />
```

9.5 ★★ El hijo recibe un setter directamente

```
type Props = { alCambiar: React.Dispatch<React.SetStateAction<string>> };
```

9.6 ★★ Callback con dos argumentos

```
type Props = { alEditar: (id: number, lema: string) => void };
```

9.7 ★★ Componente de tarjeta que borra y vota

```
type TarjetaProps = {
  candidato: Candidato;
  estado: Estado;
  alVotar: (id: number) => void;
  alEliminar?: (id: number) => void;
};
```

9.8 ★★ Ejecutar el callback opcional solo si existe

```
<button onClick={() => alEliminar?.(candidato.id)}>Eliminar</button>
```

9.9 ★★ Un efecto que se ejecuta al montar

```
useEffect(() => {
  document.title = "Elección de delegado";
}, []);
```

9.10 ★★ Efecto que se repite cuando cambia un dato

```
useEffect(() => {
  document.title = `Votos: ${totalVotos}`;
}, [totalVotos]);
```

9.11 ★★ Efecto con limpieza (temporizador)

```
useEffect(() => {
  const id = setInterval(() => setSegundos(s => s + 1), 1000);
  return () => clearInterval(id);
}, []);
```

9.12 ★★★ Escuchar un evento del navegador

```
useEffect(() => {
  const alPulsar = (e: KeyboardEvent) => { if (e.key === "Escape") setBusqueda(""); };
  window.addEventListener("keydown", alPulsar);
  return () => window.removeEventListener("keydown", alPulsar);
}, []);
```

9.13 ★★★ Guardar en `localStorage` cada vez que cambia la lista

```
useEffect(() => {
  localStorage.setItem("candidatos", JSON.stringify(candidatos));
}, [candidatos]);
```

9.14 ★★★ Leer de `localStorage` al iniciar

```
const [candidatos, setCandidatos] = useState<Candidato[]>(() => {
  const guardado = localStorage.getItem("candidatos");
  return guardado ? (JSON.parse(guardado) as Candidato[]) : candidatosIniciales;
});
```

9.15 ★★★ Cargar datos con `fetch`, cargando y error

```
const [datos, setDatos] = useState<Usuario[]>([]);
const [cargando, setCargando] = useState(true);
const [error, setError] = useState<string | null>(null);

useEffect(() => {
  let cancelado = false;
  fetch("https://jsonplaceholder.typicode.com/users")
    .then(r => { if (!r.ok) throw new Error("HTTP " + r.status); return r.json() as Promise<Usuario[]>; })
    .then(d => { if (!cancelado) setDatos(d); })
    .catch(e => { if (!cancelado) setError(e instanceof Error ? e.message : "Error"); })
    .finally(() => { if (!cancelado) setCargando(false); });
  return () => { cancelado = true; };
}, []);
```

9.16 ★★★ Lo mismo con `async/await`

```
useEffect(() => {
  async function cargar() {
    try {
      const r = await fetch("/datos.json");
      if (!r.ok) throw new Error("HTTP " + r.status);
      setDatos((await r.json()) as Usuario[]);
    } catch (e) {
      setError(e instanceof Error ? e.message : "Error");
    } finally {
      setCargando(false);
    }
  }
  cargar();
}, []);
```

9.17 ★★★ Volver a cargar cuando cambia un parámetro

```
useEffect(() => {
  fetch(`/api/productos?categoria=${categoria}`).then(r => r.json()).then(setProductos);
}, [categoria]);
```

9.18 ★★ Lo que NO debe ir en un efecto (datos derivados)

```
// Mal:
useEffect(() => setTotal(candidatos.reduce((s, c) => s + c.votos, 0)), [candidatos]);
// Bien:
const total = candidatos.reduce((s, c) => s + c.votos, 0);
```

9.19 ★★ `useRef` para guardar un valor sin provocar render

```
const contadorRenders = useRef(0);
contadorRenders.current += 1;
```

9.20 ★★★ `useMemo` para un cálculo pesado (opcional)

```
import { useMemo } from "react";
const visibles = useMemo(
  () => [...candidatos].sort(comparadores[orden]),
  [candidatos, orden]
);
```

9.21 ★★★ `useCallback` para una función estable (opcional)

```
import { useCallback } from "react";
const votar = useCallback((id: number) => {
  setCandidatos(prev => prev.map(c => (c.id === id ? { ...c, votos: c.votos + 1 } : c)));
}, []);
```

9.22 ★★★ Hook personalizado para un contador

```
function useContador(inicial = 0, max = 10) {
  const [n, setN] = useState(inicial);
  const sumar = () => setN(p => Math.min(max, p + 1));
  const restar = () => setN(p => Math.max(0, p - 1));
  const reiniciar = () => setN(inicial);
  return { n, sumar, restar, reiniciar };
}
```

9.23 ★★★ Hook personalizado para `localStorage`

```
function useAlmacen<T>(clave: string, inicial: T) {
  const [valor, setValor] = useState<T>(() => {
    const g = localStorage.getItem(clave);
    return g ? (JSON.parse(g) as T) : inicial;
  });
  useEffect(() => { localStorage.setItem(clave, JSON.stringify(valor)); }, [clave, valor]);
  return [valor, setValor] as const;
}
```

9.24 ★★★ `useReducer` con acciones tipadas

```
type Accion =
  | { tipo: "votar"; id: number }
  | { tipo: "anadir"; candidato: Candidato }
  | { tipo: "reiniciar" };

function reductor(estado: Candidato[], accion: Accion): Candidato[] {
  switch (accion.tipo) {
    case "votar":
      return estado.map(c => (c.id === accion.id ? { ...c, votos: c.votos + 1 } : c));
    case "anadir":
      return [...estado, accion.candidato];
    case "reiniciar":
      return estado.map(c => ({ ...c, votos: 0 }));
  }
}
const [lista, despachar] = useReducer(reductor, candidatosIniciales);
// despachar({ tipo: "votar", id: 3 });
```

9.25 ★★★ Contexto tipado (compartir sin pasar props nivel a nivel)

```
import { createContext, useContext } from "react";
type Ctx = { candidatos: Candidato[]; votar: (id: number) => void };
const Contexto = createContext<Ctx | null>(null);
function useElecciones(): Ctx {
  const c = useContext(Contexto);
  if (!c) throw new Error("Falta el proveedor");
  return c;
}
```

---

10. Variantes del ensayo "Elección de delegado/a"

Aquí cambias un detalle del enunciado. Es justo el tipo de cambio que puede aparecer en un examen.

Apartado 1 · modelo y `obtenerEstado`

10.1 ★ Añadir un campo `curso` (texto) a `Candidato`

```
export type Candidato = { id: number; nombre: string; numeroLista: number; votos: number; lema?: string;
    curso: string };
```

10.2 ★ Añadir un campo opcional `foto?: string`

```
export type Candidato = { /* … */ foto?: string };
```

10.3 ★★ Estado nuevo `"empate"` (varios líderes a la vez)

```
export type Estado = "ninguno" | "algunos" | "lider" | "empate";
export function obtenerEstado(votos: number, maximo: number, lideres: number): Estado {
  if (votos === 0) return "ninguno";
  if (votos === maximo) return lideres > 1 ? "empate" : "lider";
  return "algunos";
}
// en App: const lideres = candidatos.filter(c => c.votos === maximo && maximo > 0).length;
```

10.4 ★★ Estado `"ultimo"` para el que menos votos tiene (con votos)

```
export function obtenerEstado(votos: number, maximo: number, minimo: number): "ninguno" | "lider" | "ultimo"
    | "algunos" {
  if (votos === 0) return "ninguno";
  if (votos === maximo) return "lider";
  if (votos === minimo) return "ultimo";
  return "algunos";
}
```

10.5 ★ Pruebas de `obtenerEstado`

```
console.assert(obtenerEstado(0, 5) === "ninguno");
console.assert(obtenerEstado(0, 0) === "ninguno");
console.assert(obtenerEstado(5, 5) === "lider");
console.assert(obtenerEstado(3, 5) === "algunos");
console.assert(obtenerEstado(1, 1) === "lider");
```

Apartado 2 · tarjeta

10.6 ★ Mostrar también el porcentaje de votos

```
type Props = { /* … */ porcentaje: number };
<p>{porcentaje} % de los votos</p>
```

10.7 ★★ Mostrar la posición («1.º», «2.º»…)

```
type Props = { /* … */ posicion: number };
<span className="posicion">{posicion}.º</span>
```

10.8 ★★ Mensaje distinto para cada estado con una tabla

```
const mensaje: Record<Estado, string> = {
  ninguno: "Todavía sin votos",
  algunos: "",
  lider: "♛ Va en cabeza",
};
{mensaje[estado] && <p>{mensaje[estado]}</p>}
```

10.9 ★ Botón «Quitar voto» deshabilitado en 0

```
<button onClick={() => alQuitar(candidato.id)} disabled={candidato.votos === 0}>−</button>
```

10.10 ★★ Mostrar «1 voto» / «N votos»

```
<p>{candidato.votos} {candidato.votos === 1 ? "voto" : "votos"}</p>
```

10.11 ★★ Barra de progreso proporcional a los votos

```
<progress value={candidato.votos} max={Math.max(1, maximo)} />
```

10.12 ★★ Mostrar el lema entre comillas y cursiva, solo si existe

```
{candidato.lema && <p><em>«{candidato.lema}»</em></p>}
```

Apartado 3 · lista

10.13 ★★ Lista ordenada por votos (el líder arriba)

```
const ordenados = [...candidatos].sort((a, b) => b.votos - a.votos);
```

10.14 ★★ Lista ordenada por nombre A–Z

```
const ordenados = [...candidatos].sort((a, b) => a.nombre.localeCompare(b.nombre, "es"));
```

10.15 ★★ Orden elegido por el usuario con un `select`

```
const [orden, setOrden] = useState<Orden>("lista-asc");
const visibles = [...candidatos].sort(comparadores[orden]);
```

10.16 ★★ Mostrar quién va ganando (nombre del líder)

```
const lider = candidatos.find(c => c.votos === maximo && maximo > 0);
{lider && <p>Va ganando: {lider.nombre}</p>}
```

10.17 ★★ Mostrar varios líderes si hay empate

```
const lideres = candidatos.filter(c => maximo > 0 && c.votos === maximo);
<p>{lideres.map(c => c.nombre).join(" y ")}</p>
```

10.18 ★★ Mostrar el número de candidatos y de candidatos sin votos

```
<p>Candidatos: {candidatos.length} · Sin votos: {candidatos.filter(c => c.votos === 0).length}</p>
```

10.19 ★★ Ocultar los candidatos sin votos con un `checkbox`

```
const [soloConVotos, setSoloConVotos] = useState(false);
const visibles = soloConVotos ? candidatos.filter(c => c.votos > 0) : candidatos;
```

10.20 ★★ Filtrar por estado con un `select`

```
const [filtroEstado, setFiltroEstado] = useState<Estado | "todos">("todos");
const visibles = candidatos.filter(c =>
  filtroEstado === "todos" || obtenerEstado(c.votos, maximo) === filtroEstado
);
```

10.21 ★★ Buscador por nombre y por lema

```
const q = busqueda.trim().toLowerCase();
const visibles = candidatos.filter(c =>
  c.nombre.toLowerCase().includes(q) || (c.lema ?? "").toLowerCase().includes(q)
);
```

10.22 ★★ Mensaje distinto si el filtro no encuentra nada

```
{candidatos.length === 0 && <p className="vacio">Todavía no hay candidaturas.</p>}
{candidatos.length > 0 && visibles.length === 0 && <p className="vacio">Ningún resultado.</p>}
```

Apartado 4 · votar

10.23 ★★ Votar solo si no se ha votado ya (un voto por persona)

```
const [yaVoto, setYaVoto] = useState(false);
const votar = (id: number) => {
  if (yaVoto) return;
  setCandidatos(prev => prev.map(c => (c.id === id ? { ...c, votos: c.votos + 1 } : c)));
  setYaVoto(true);
};
```

10.24 ★★★ Recordar a quién has votado (un solo voto, cambiable)

```
const [votado, setVotado] = useState<number | null>(null);
const votar = (id: number) => {
  setCandidatos(prev => prev.map(c => {
    if (c.id === id) return { ...c, votos: c.votos + 1 };
    if (c.id === votado) return { ...c, votos: Math.max(0, c.votos - 1) };
    return c;
  }));
  setVotado(id);
};
```

10.25 ★★ Reiniciar toda la votación

```
const reiniciar = () => setCandidatos(prev => prev.map(c => ({ ...c, votos: 0 })));
```

10.26 ★★ Confirmar antes de borrar

```
const eliminar = (id: number) => {
  if (!window.confirm("¿Seguro que quieres eliminar la candidatura?")) return;
  setCandidatos(prev => prev.filter(c => c.id !== id));
};
```

Apartado 5 · formulario

10.27 ★ Cambiar el rango válido a 1–40

```
const valido = nombre.trim() !== "" && numero !== "" && Number(numero) >= 1 && Number(numero) <= 40;
```

10.28 ★★ Exigir que el número sea entero

```
const n = Number(numero);
const valido2 = nombre.trim() !== "" && Number.isInteger(n) && n >= 1 && n <= 30;
```

10.29 ★★ Hacer el lema obligatorio

```
const valido3 = nombre.trim() !== "" && lema.trim() !== "" && Number(numero) >= 1 && Number(numero) <= 30;
```

10.30 ★★ Nombre de al menos 3 caracteres

```
const valido4 = nombre.trim().length >= 3 && Number(numero) >= 1 && Number(numero) <= 30;
```

10.31 ★★ Nombre sin números

```
const sinDigitos = !/\d/.test(nombre);
```

10.32 ★★ Normalizar el nombre (primera letra en mayúscula)

```
const normalizar = (s: string) => s.trim().replace(/\s+/g, " ").split(" ").map(p => p[0].toUpperCase() +
    p.slice(1).toLowerCase()).join(" ");
```

10.33 ★★ Añadir al principio de la lista en vez de al final

```
const anadir = (c: Candidato) => setCandidatos(prev => [c, ...prev]);
```

10.34 ★★ Añadir y ordenar de golpe por número de lista

```
const anadir = (c: Candidato) =>
  setCandidatos(prev => [...prev, c].sort((a, b) => a.numeroLista - b.numeroLista));
```

10.35 ★★ Id sin colisiones con `crypto.randomUUID()`

```
export type Candidato = { id: string; /* … */ };
const nuevo = { id: crypto.randomUUID(), /* … */ };
```

10.36 ★★ Id con `useRef` como contador

```
const siguienteId = useRef(100);
const nuevo = { id: siguienteId.current++, /* … */ };
```

10.37 ★★★ Formulario que sirve para añadir y editar

```
type Props = { inicial?: Candidato; alGuardar: (c: Candidato) => void };
const [nombre, setNombre] = useState(inicial?.nombre ?? "");
const [numero, setNumero] = useState(inicial ? String(inicial.numeroLista) : "");
const [lema, setLema] = useState(inicial?.lema ?? "");
// al guardar: id: inicial?.id ?? Date.now(), votos: inicial?.votos ?? 0
```

10.38 ★★ Mostrar un mensaje de éxito tras añadir

```
const [mensaje, setMensaje] = useState<string | null>(null);
// al enviar: setMensaje(`Candidatura de ${nombre.trim()} añadida`);
{mensaje && <p role="status">{mensaje}</p>}
```

---

11. Encuentra el error (y corrígelo)

> Intenta localizarlos antes de mirar la corrección.

11.1 ★ Muta el estado

```
// ✗
const anadir = () => { candidatos.push(nuevo); setCandidatos(candidatos); };
// ✓
const anadir = () => setCandidatos(prev => [...prev, nuevo]);
```

11.2 ★ Ejecuta al renderizar

```
// ✗ <button onClick={votar(c.id)}>
// ✓ <button onClick={() => votar(c.id)}>
```

11.3 ★ Estado sin tipo

```
// ✗ const [lista, setLista] = useState([]);   // never[]
// ✓ const [lista, setLista] = useState<Candidato[]>([]);
```

11.4 ★ Falta `key`

```
// ✗ {candidatos.map(c => <li>{c.nombre}</li>)}
// ✓ {candidatos.map(c => <li key={c.id}>{c.nombre}</li>)}
```

11.5 ★ `value` sin `onChange`

```
// ✗ <input value={nombre} />
// ✓ <input value={nombre} onChange={e => setNombre(e.target.value)} />
```

11.6 ★ Formulario que recarga la página

```
// ✗ const enviar = () => { … }
// ✓ const enviar = (e: FormEvent<HTMLFormElement>) => { e.preventDefault(); … }
```

11.7 ★ Dato derivado guardado en estado

```
// ✗ const [total, setTotal] = useState(0);   // y luego sincronizarlo a mano
// ✓ const total = candidatos.reduce((s, c) => s + c.votos, 0);
```

11.8 ★★ Reasignar una constante de estado

```
// ✗ contador = contador + 1;
// ✓ setContador(prev => prev + 1);
```

11.9 ★★ `sort` sobre el estado

```
// ✗ const visibles = candidatos.sort((a, b) => b.votos - a.votos);   // muta el estado
// ✓ const visibles = [...candidatos].sort((a, b) => b.votos - a.votos);
```

11.10 ★★ Ordenar números sin comparador

```
// ✗ precios.sort()
// ✓ [...precios].sort((a, b) => a - b)
```

11.11 ★★ `reverse` sobre el estado

```
// ✗ const alReves = lista.reverse();
// ✓ const alReves = [...lista].reverse();
```

11.12 ★★ Comparar texto con `-`

```
// ✗ [...productos].sort((a, b) => a.nombre - b.nombre)   // error de tipos
// ✓ [...productos].sort((a, b) => a.nombre.localeCompare(b.nombre, "es"))
```

11.13 ★★ Comparador que devuelve booleano

```
// ✗ lista.sort((a, b) => a.precio > b.precio)   // debe devolver un número
// ✓ lista.sort((a, b) => a.precio - b.precio)
```

11.14 ★★ Se usa un número como texto

```
// ✗ const n: number = e.target.value;
// ✓ const n: number = Number(e.target.value);
```

11.15 ★★ Opcional usado sin comprobar

```
// ✗ const l = candidato.lema.length;   // lema puede ser undefined
// ✓ const l = candidato.lema?.length ?? 0;
```

11.16 ★★ `&&` pinta un 0

```
// ✗ {candidatos.length && <Lista />}
// ✓ {candidatos.length > 0 && <Lista />}
```

11.17 ★★ Tres `setX(x + 1)` seguidos

```
// ✗ setN(n + 1); setN(n + 1); setN(n + 1);   // sube 1
// ✓ setN(p => p + 1); setN(p => p + 1); setN(p => p + 1);   // sube 3
```

11.18 ★★ `setState` en el cuerpo del componente

```
// ✗ function C() { const [n, setN] = useState(0); setN(1); … }   // bucle infinito
// ✓ llámalo desde un evento o un useEffect con condiciones
```

11.19 ★★ Efecto sin array de dependencias

```
// ✗ useEffect(() => { setDatos(...) });          // se ejecuta tras cada render
// ✓ useEffect(() => { … }, []);
```

11.20 ★★ Falta importar el tipo

```
// ✗ const f = (e: ChangeEvent<HTMLInputElement>) => …   // sin import
// ✓ import { useState, type ChangeEvent } from "react";
```

11.21 ★★ Objeto con la propiedad opcional mal escrita

```
// ✗ const c: Candidato = { id: 1, nombre: "Ana", numeroLista: 3, votos: 0, lema: undefined as string };
// ✓ const c: Candidato = { id: 1, nombre: "Ana", numeroLista: 3, votos: 0 };
```

11.22 ★★ Un cambio parcial que borra el resto de campos

```
// ✗ setDatos({ nombre: "Luis" });                 // se pierden los demás campos
// ✓ setDatos(prev => ({ ...prev, nombre: "Luis" }));
```

11.23 ★★ Modificar un objeto dentro de `map`

```
// ✗ prev.map(c => { if (c.id === id) c.votos++; return c; })   // muta el objeto
// ✓ prev.map(c => (c.id === id ? { ...c, votos: c.votos + 1 } : c))
```

11.24 ★★ Comparar con `==` en vez de `===`

```
// ✗ if (id == "3")
// ✓ if (id === 3)
```

11.25 ★★ `Math.max` con un array vacío

```
// ✗ Math.max(...[])            // -Infinity
// ✓ Math.max(0, ...lista.map(c => c.votos))
```

---

12. Mini-simulacros con tiempo

Cronométrate. Sin mirar soluciones. Al terminar, comprueba con `npx tsc --noEmit` y en el navegador.

Simulacro A (25 min): «Tienda»

1. Define `Producto` y `ItemCarrito`.
2. Muestra los productos de `productos` ordenados por precio ascendente.
3. Un `select` cambia el orden entre: precio ↑, precio ↓, nombre A–Z, nombre Z–A, categoría.
4. Un `input` filtra por nombre sin distinguir mayúsculas.
5. Un `select` filtra por categoría (con la opción «Todas»).
6. Botón «Añadir» que mete el producto al carrito o suma cantidad.
7. Muestra el total y el número de unidades (derivados).
8. Botón «Quitar» que resta cantidad y elimina la línea al llegar a 0.
9. Los productos sin stock tienen el botón deshabilitado.
10. Si el carrito está vacío, muestra «Carrito vacío».

Simulacro B (25 min): «Tareas»

1. `Tarea = { id; texto; hecha; prioridad: "alta" | "media" | "baja" }`.
2. Formulario con texto y `select` de prioridad; no se envía si el texto está vacío.
3. Lista con casilla para marcar como hecha y botón de borrar.
4. Contador de pendientes (derivado).
5. Botones de filtro: todas / pendientes / hechas.
6. Ordenar por prioridad (alta → baja) y, dentro de cada una, por texto.
7. Botón «Borrar hechas».
8. Barra de progreso con el porcentaje de hechas.
9. Guardar en `localStorage`.
10. Mensaje «No hay tareas» si la lista visible está vacía.

Simulacro C (25 min): «Delegado, versión 2»

1. Añade el campo `curso` al candidato y al formulario (obligatorio).
2. Ordena las tarjetas por votos descendente, con desempate por nombre.
3. Muestra el porcentaje de votos de cada candidato.
4. Si hay empate en cabeza, todos los empatados muestran «= Empate».
5. Botón «Quitar voto» (nunca por debajo de 0).
6. Buscador por nombre o lema.
7. Un solo voto por persona (se puede cambiar de candidato).
8. Botón «Eliminar» con confirmación.
9. No permitas dos candidatos con el mismo número de lista.
10. Muestra el ganador solo si el total de votos es mayor que 0.

13. Chuleta de ordenación (para tenerla a mano)

Quiero ordenar…	Código (sobre una copia)
Números ↑	`[...x].sort((a, b) => a - b)`
Números ↓	`[...x].sort((a, b) => b - a)`
Texto A→Z	`[...x].sort((a, b) => a.localeCompare(b, "es"))`
Texto Z→A	`[...x].sort((a, b) => b.localeCompare(a, "es"))`
Texto ignorando tildes y mayúsculas	`a.localeCompare(b, "es", { sensitivity: "base" })`
Texto con números dentro	`a.localeCompare(b, "es", { numeric: true })`
Objetos por campo numérico ↑	`(a, b) => a.precio - b.precio`
Objetos por campo numérico ↓	`(a, b) => b.precio - a.precio`
Objetos por campo de texto	`(a, b) => a.nombre.localeCompare(b.nombre, "es")`
Dos criterios	`(a, b) => crit1 || crit2`
Booleano (falsos primero)	`(a, b) => Number(a.hecha) - Number(b.hecha)`
Booleano (verdaderos primero)	`(a, b) => Number(b.hecha) - Number(a.hecha)`
Fecha ISO ↑	`(a, b) => a.fecha.localeCompare(b.fecha)`
Fecha ↓	`(a, b) => b.fecha.localeCompare(a.fecha)`
Orden personalizado	`(a, b) => orden.indexOf(a.x) - orden.indexOf(b.x)`
Opcional (undefined al final)	`(a, b) => (a.x ?? Infinity) - (b.x ?? Infinity)`
Invertir un orden	`.reverse()` sobre una copia, o cambia `a` y `b`

Recuerda: el comparador devuelve un **número**: negativo si `a` va antes, positivo si va después, 0 si da igual. Nunca un booleano.

Fin del banco. Cuando lo termines, dime cuáles te han costado más y te preparo una segunda ronda centrada en esas.